

AI-BASED VIRTUAL CALL ASSISTANT FOR INTELLIGENT INCOMING CALL MANAGEMENT

Final Year Project — Technical Documentation

Recommended Technology Architecture

React + Node.js + Express + Python + FastAPI + LangGraph + LLM + Plivo + PostgreSQL

Project Type: AI-powered incoming call screening and personal voice assistant

Document scope: Requirements, architecture, implementation plan, database, APIs, testing, security, and 45-day schedule

1. Executive Summary

This project proposes an AI-powered virtual call assistant that automatically handles incoming phone calls when the user is unavailable. The assistant answers the call, identifies itself as an AI assistant, asks the caller for their name and purpose, asks a concise follow-up question when required, detects intent and urgency, informs the caller that the user is currently unavailable, and ends the call politely.

After the call, the system stores the call information, recording, transcript, and AI-generated summary. A web dashboard allows the user to review handled calls, listen to recordings, read transcripts, see the purpose and priority, and initiate a callback.

Core architecture: React provides the dashboard; Node.js and Express provide the main application backend and telephony webhooks; a separate Python FastAPI service runs LangGraph; LangGraph manages the stateful AI workflow; an LLM provides language understanding and response generation; Plivo provides telephony; PostgreSQL stores structured data; and object storage holds audio recordings.

Important terminology: LangGraph is not the AI model. It is the agent orchestration/state-management framework. The LLM is the underlying language model.

The recommended development strategy is incremental: first build and test the LangGraph agent with text, then expose it through FastAPI, connect Node.js to the AI service, add telephony and speech, and finally complete recording, dashboard, notifications, and callback features.

2. Project Definition

Recommended title: AI-Based Virtual Call Assistant for Intelligent Incoming Call Management

Alternative title: AI-Powered Intelligent Call Screening and Personal Voice Assistant

One-line description: An AI-powered virtual assistant that automatically handles incoming calls when the user is unavailable, identifies the caller and purpose of the call, conducts a short conversation, records and transcribes the call, generates an AI summary, detects urgency, and notifies the user through a web dashboard.

3. Problem Statement

People frequently miss important calls while attending classes, studying, working, driving, sleeping, or otherwise being unavailable. Traditional missed-call systems provide limited information, while voicemail generally records a message without intelligently understanding the caller's intent, determining urgency, or presenting structured information.

The proposed system acts as a personal virtual receptionist. When the user is busy, the AI answers the call, understands the caller's purpose, communicates the user's unavailability, and creates a structured record that the user can review later.

4. Objectives

- Automatically handle incoming calls when the user is busy.
- Introduce the system honestly as an AI assistant.
- Identify the caller and purpose of the call.
- Ask relevant follow-up questions when necessary.
- Detect intent and apparent urgency.
- Inform callers that the user is unavailable without impersonating the user.
- Record and transcribe conversations.

- Generate concise AI summaries and callback requirements.
- Securely store call information and recordings.
- Notify the user after a handled call.
- Provide a React dashboard for call history and details.
- Allow recording playback, transcript viewing, and callback initiation.

5. Scope

In scope: incoming call handling, AI conversation, caller/purpose extraction, urgency detection, recording, speech-to-text, summary generation, dashboard, notifications, authentication, call history, and callback.

Out of scope for the MVP: training a foundation model, building a telecom network, custom speech-recognition/TTS models, complex multi-agent systems, and extensive third-party integrations.

Future scope: multilingual/Hindi/Hinglish support, contact recognition, advanced spam detection, sentiment analysis, calendar integration, smart callback scheduling, WhatsApp integration, appointment scheduling, and enterprise receptionist mode.

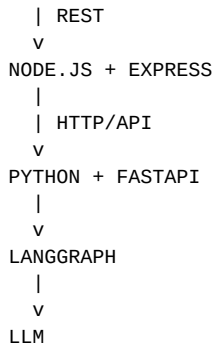
6. Final Technical Stack

Layer	Technology	Purpose
Frontend	React.js	Web dashboard and user interface
Backend	Node.js + Express.js	REST APIs, application logic, authentication, webhooks
Backend language	JavaScript/TypeScript	Main application development
AI service	Python + FastAPI	Expose the LangGraph agent as a service
Agent framework	LangGraph	Conversation state, workflow, routing and agent orchestration
LLM	OpenAI API or suitable LLM	Natural-language reasoning and response generation
STT	Whisper / speech-to-text API	Caller voice to text
TTS	TTS API	AI response text to speech
Telephony	Plivo	Phone number, inbound calls, call control and audio integration
Database	PostgreSQL	Users, calls, transcripts, summaries, notifications
Audio storage	Object/cloud storage	Call recordings
Authentication	JWT + password hashing	Secure user authentication
Testing	Postman + automated tests	API and integration testing
Version control	Git + GitHub	Source control and collaboration
Deployment	Render/Railway/AWS or equivalent	Hosting

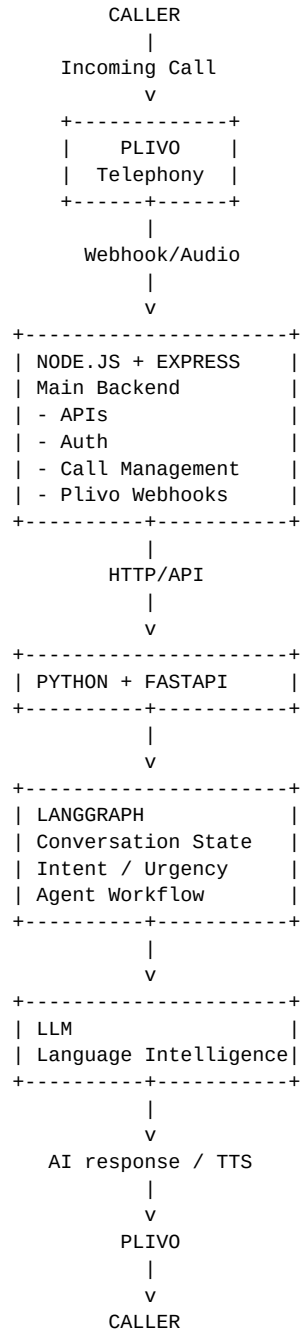
7. Why This Architecture

The architecture intentionally separates the application backend from the AI agent. Node.js/Express is responsible for normal application concerns such as authentication, APIs, database access and telephony webhooks. Python/FastAPI is dedicated to the AI workflow. This lets the project use LangGraph in its Python ecosystem without forcing the entire application to Python.

REACT
|

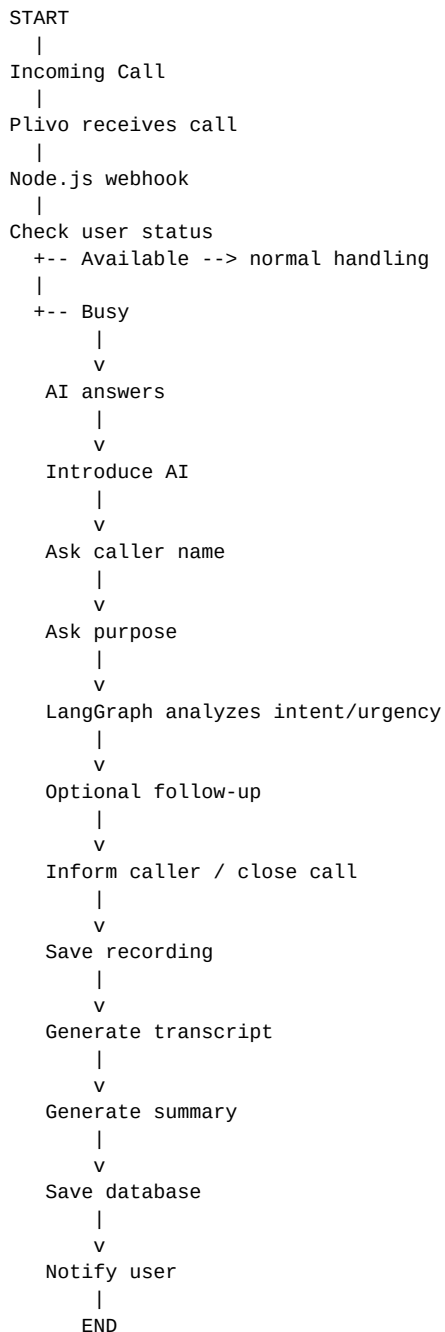


8. High-Level System Architecture



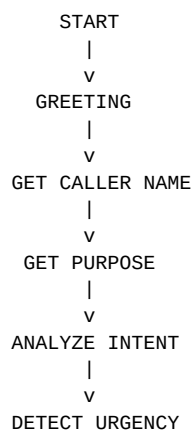
PostgreSQL <--- Node.js ---> React Dashboard
 Audio Storage <--- Node.js

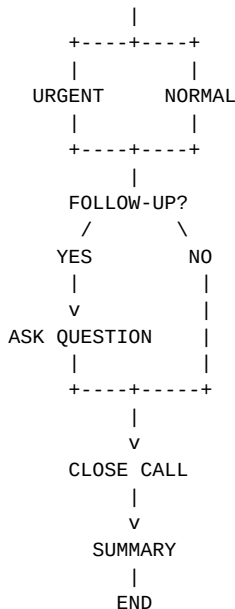
9. Complete Call Flow



10. LangGraph Agent Design

The LangGraph service is the stateful conversational controller. It should not be described as the AI model. The LLM supplies language understanding and generation, while LangGraph controls the sequence, state and decisions.





11. Agent State

```

class CallState(TypedDict):
    call_id: str
    caller_number: str
    caller_name: str
    purpose: str
    urgency: str
    conversation: list
    summary: str
    callback_required: bool

```

Example structured output:

```

{
  "caller_name": "Rahul Sharma",
  "purpose": "Discuss final year project",
  "urgency": "normal",
  "summary": "Rahul wants to discuss the final year project and requested a callback.",
  "callback_required": true
}

```

12. Recommended AI Conversation Policy

The agent should be concise, professional and transparent. It must not impersonate the user or make promises on the user's behalf.

System policy:

You are a professional personal AI phone assistant.

Your objectives are:

1. Politely answer the incoming call.
2. Introduce yourself as an AI assistant.
3. Ask for the caller's name.
4. Ask for the purpose of the call.
5. Ask a short follow-up question if necessary.
6. Determine whether the call appears urgent.
7. Do not pretend to be the user.
8. Do not make promises on behalf of the user.
9. Tell the caller that the user is currently unavailable.
10. Tell the caller that the user will call them back.
11. Keep the conversation concise.
12. End the call politely.

At the end, produce:

- caller_name

- purpose
- urgency
- summary
- callback_required

13. Example Conversation

AI: Hello, you've reached the AI assistant. The person you're trying to reach is currently unavailable. May I know your name?

Caller: Hi, I'm Rahul.

AI: Thank you, Rahul. Could you please tell me the purpose of your call?

Caller: I want to discuss our final year project.

AI: Sure. Is there anything specific you'd like me to pass along?

Caller: Yes, I want to discuss tomorrow's presentation.

AI: Thank you. I've noted that. They are currently busy and will call you back when they're available.

14. Functional Modules

Module	Main responsibilities
User Management	Registration, login, profile, authentication, busy/available status
Telephony	Incoming calls, answering, call ending, recording, caller number
AI Conversation	Greeting, caller name, purpose, follow-up, closing
AI Analysis	Intent, urgency, callback requirement, summary
Speech	STT and TTS integration
Call Management	Call history, duration, status, recording references
Dashboard	Recent calls, details, transcript, summary, playback, callback
Notifications	New-call and urgent-call notifications

15. Database Design

Large audio files should not be stored directly in PostgreSQL. Store the recording in object/cloud storage and keep only a secure URL/reference in the database.

Users table

```
users
-----
id
name
email
phone_number
password_hash
busy_status
created_at
updated_at
```

Calls table

```
calls
-----
id
user_id
```

```

caller_number
caller_name
start_time
end_time
duration
status
purpose
priority
summary
recording_url
callback_required
created_at

```

Transcripts table

```

transcripts
-----
id
call_id
speaker
text
timestamp

```

Notifications table

```

notifications
-----
id
user_id
call_id
title
message
is_read
created_at

```

16. REST API Design

Method	Endpoint	Purpose
POST	/api/auth/register	Register user
POST	/api/auth/login	Login
GET	/api/auth/me	Current user
GET	/api/users/profile	User profile
PATCH	/api/users/status	Busy/available status
POST	/api/calls/incoming	Telephony webhook
GET	/api/calls	Call history
GET	/api/calls/:id	Call details
GET	/api/calls/:id/transcript	Transcript
GET	/api/calls/:id/recording	Recording access
POST	/api/calls/:id/callback	Initiate callback
POST	/agent/message	LangGraph message processing
POST	/agent/summary	Summary generation

17. Node ↔ Python Communication

Node.js sends conversation turns to the Python FastAPI service. The Python service runs the LangGraph workflow and returns the generated response plus structured call metadata.

POST /agent/message

```
{
  "call_id": "CALL123",
  "message": "I want to discuss tomorrow's presentation."
}
```

Response:

```
{
  "response": "Sure. Is there anything specific you would like me to pass along?",
  "purpose": "Final year project",
  "urgency": "normal",
  "callback_required": true
}
```

18. React Dashboard

AI CALL ASSISTANT

Status: BUSY

NEW CALL
Rahul Sharma
Project Discussion
01:42

[VIEW CALL]

RECENT CALLS

Rahul Sharma	Project	01:42
Amit Kumar	Meeting	00:58
Priya Sharma	Personal	02:11

Call details should show caller, phone number, time, duration, priority, purpose, AI summary, recording player, transcript, and callback action.

19. Recommended Repository Structure

```
AI-CALLER-AGENT/
|
+-- frontend/
|   +-- src/
|       +-- components/
|       +-- pages/
|       +-- services/
|       +-- hooks/
|       +-- App.jsx
|
+-- backend/
|   +-- src/
|       +-- controllers/
|       +-- routes/
|       +-- middleware/
|       +-- services/
|       +-- models/
|       +-- database/
|       +-- server.js
|
+-- ai-agent/
|   +-- app/
|       +-- main.py
|       +-- graph.py
|       +-- state.py
|       +-- prompts.py
|       +-- nodes/
|           +-- greeting.py
|           +-- caller.py
|           +-- intent.py
|           +-- urgency.py
|           +-- summary.py
```

```
|   +-- requirements.txt
|
+-- docs/
|   +-- architecture/
|   +-- diagrams/
|   +-- api/
|   +-- report/
|
+-- .gitignore
+-- README.md
```

20. 45-Day Development Plan

Week 1 — Planning & environment

- Day 1: Finalize title, problem statement, objectives, scope and feature list.
- Day 2: Finalize architecture, technology stack and diagrams.
- Day 3: Create GitHub repository and project folders.
- Day 4: Set up React and create dashboard skeleton.
- Day 5: Set up Node.js + Express and health endpoint.
- Day 6: Set up PostgreSQL and initial schema.
- Day 7: Connect React → Express → PostgreSQL. Milestone: basic full-stack app.

Week 2 — LangGraph AI

- Day 8: Create Python environment and install LangGraph, LLM SDK, FastAPI and Pydantic.
- Day 9: Implement basic LLM interaction.
- Day 10: Create LangGraph state.
- Day 11: Implement greeting node.
- Day 12: Implement caller-name node.
- Day 13: Implement purpose node.
- Day 14: Implement closing node. Milestone: text-based AI conversation.

Week 3 — Intelligent agent

- Day 15: Add intent detection.
- Day 16: Add urgency detection.
- Day 17: Add callback-required detection.
- Day 18: Add conversation state/memory.
- Day 19: Add summary generation.
- Day 20: Return structured JSON.
- Day 21: Test 20–30 simulated conversations. Milestone: AI agent functionally complete.

Week 4 — Node ↔ Python

- Day 22: Create FastAPI AI service.
- Day 23: Create POST /agent/message.

- Day 24: Connect Node → Python.
- Day 25: Test React → Node → Python → LangGraph → LLM.
- Day 26: Add error handling.
- Day 27: Add service authentication.
- Day 28: Improve prompts and responses. Milestone: integrated AI backend.

Week 5 — Telephony

- Day 29: Configure Plivo account/number according to current availability.
- Day 30: Configure inbound-call webhook.
- Day 31: Make Node answer an incoming call.
- Day 32: Connect voice input.
- Day 33: Integrate speech-to-text.
- Day 34: Connect STT → LangGraph.
- Day 35: Connect LangGraph → TTS → phone. Milestone: real phone conversation.

Week 6 — Recording & data

- Day 36: Implement call recording.
- Day 37: Store recordings in object/cloud storage.
- Day 38: Store call metadata.
- Day 39: Store transcript.
- Day 40: Generate and store summary. Milestone: complete call record.

Week 7 — Dashboard & final integration

- Day 41: Build call-history page.
- Day 42: Build call-details page.
- Day 43: Add recording playback, transcript, summary and priority.
- Day 44: Add callback functionality.
- Day 45: Full integration, bug fixing and final demonstration.

21. Testing Plan

ID	Test	Expected result
T01	Incoming call	AI answers
T02	Caller gives name	Name extracted correctly
T03	Caller gives purpose	Purpose identified
T04	Unclear purpose	AI asks follow-up
T05	Caller says urgent	High priority
T06	Normal call	Normal priority
T07	Call ends	Recording saved

ID	Test	Expected result
T08	Transcript generation	Transcript available
T09	Summary generation	Accurate concise summary
T10	Notification	User receives notification
T11	Dashboard	Call appears in history
T12	Recording playback	Audio plays
T13	Transcript view	Conversation displayed
T14	Callback	Callback initiated
T15	Authentication	Unauthorized access blocked

22. Edge Cases

- Caller is silent.
- Caller gives no name.
- Caller refuses to explain the purpose.
- Caller interrupts the AI.
- Caller speaks for a long time.
- Caller changes the purpose during the call.
- Caller explicitly says the matter is urgent.
- Unknown or suspicious number.
- Wrong number.
- Speech-to-text fails.
- LLM/API timeout.
- Telephony connection drops.
- Recording upload fails.
- Duplicate webhook/event is received.

23. Security and Privacy

The system processes sensitive communication data including phone numbers, voice recordings and transcripts. Security and privacy must therefore be treated as first-class requirements.

- Use HTTPS for all production communication.
- Hash passwords using a strong password-hashing algorithm.
- Use JWT or an equivalent secure authentication mechanism.
- Protect internal Node-to-Python API calls.
- Keep API keys and database credentials in environment variables.
- Never commit secrets to GitHub.
- Use access-controlled recording URLs.

- Restrict call data to the authenticated user.
- Implement a data-retention and deletion policy.
- Provide appropriate call-recording disclosure/consent where required.
- Follow applicable telecom, privacy and recording laws.
- Log security-relevant events without storing unnecessary sensitive data.

24. Reliability and Error Handling

- Use request timeouts between Node and Python.
- Retry transient external API failures carefully.
- Persist call state so a process restart does not destroy the conversation.
- Validate all webhook payloads.
- Handle duplicate telephony events idempotently.
- Use structured logs and correlation IDs such as `call_id`.
- Provide a safe fallback response if the AI service is unavailable.
- Do not claim that the user received a message until notification delivery is confirmed.

25. Evaluation Metrics

Metric	What to measure
Call answer success	Percentage of incoming calls successfully answered
Conversation completion	Percentage of calls reaching a valid closing
Caller-name accuracy	Correct extraction of caller identity
Purpose accuracy	Correct identification of call purpose
Urgency accuracy	Correct classification of urgent/normal cases
Summary quality	Human-rated factual and concise summary
Transcript quality	Speech-to-text accuracy on test calls
Latency	Time from caller speech to AI response
System reliability	Failed calls/API requests
Dashboard correctness	Call data matches the actual call

26. MVP vs Advanced Features

MVP — must work	Advanced — after MVP
Incoming call handling	Known caller recognition
AI greeting	Spam detection
Name + purpose	Sentiment analysis
Follow-up question	Hindi/Hinglish/multilingual
Closing	Calendar integration

MVP — must work	Advanced — after MVP
Recording	Smart callback scheduling
Transcript	WhatsApp integration
Summary	Appointment scheduling
Dashboard	Enterprise receptionist mode
Notification	Multiple simultaneous calls

27. Suggested FYP Report Structure

Chapter	Contents
1. Introduction	Background, problem, motivation, solution, objectives, scope, limitations
2. Literature Review	Voicemail, IVR, conversational AI, voice assistants, STT, TTS, LLMs, AI agents, research gap
3. System Analysis	Existing/proposed system, functional and non-functional requirements, use cases
4. System Design	Architecture, use-case, sequence, activity, ER, DFD, database and agent workflow
5. Implementation	React, Node/Express, Python/FastAPI, LangGraph, LLM, STT, TTS, Plivo, PostgreSQL
6. Testing & Results	Test cases, screenshots, performance, AI evaluation, error handling
7. Conclusion	Achievements, limitations and future scope

28. Suggested Viva Explanation

A concise technical explanation is: “**The frontend is implemented using React. Node.js and Express form the main backend and handle authentication, REST APIs, database operations and telephony webhooks. A separate Python FastAPI service hosts the LangGraph agent. LangGraph manages the stateful conversation workflow, while an LLM performs natural-language reasoning and response generation. Plivo connects the application to the telephone network. PostgreSQL stores structured call information, and recordings are stored in object storage with references in the database.**”

29. Key Technical Clarification

Do not describe LangGraph as the AI model. Use the following terminology throughout the report and viva: **LLM = language intelligence; LangGraph = agent orchestration and state management; FastAPI = Python AI-service API; Express = main application backend; Plivo = telephony layer; STT = voice-to-text; TTS = text-to-voice.**

30. Final Demo Script

- Open the dashboard and set the user status to BUSY.
- Call the configured AI phone number from another phone.
- Show that the AI answers automatically.
- Demonstrate caller-name collection.
- Demonstrate purpose collection.
- Give a normal project-related purpose.
- Let the AI close the call politely.
- Show the call ending and recording being saved.

- Refresh/open the dashboard and show the new call.
- Play the recording.
- Open the transcript.
- Show the AI summary and urgency.
- Demonstrate callback functionality.

31. Final Project Checklist

- React dashboard works.
- Node/Express API works.
- Python FastAPI service works.
- LangGraph graph works.
- LLM integration works.
- STT works.
- TTS works.
- Telephony number and inbound call flow work.
- Recording is saved.
- Transcript is saved.
- Summary is generated.
- Urgency is stored.
- Notification is delivered.
- Authentication is enforced.
- Call history is visible.
- Recording playback works.
- Callback works.
- Error handling is tested.
- Security/privacy requirements are documented.
- Final demo script has been rehearsed.

32. Development Priority

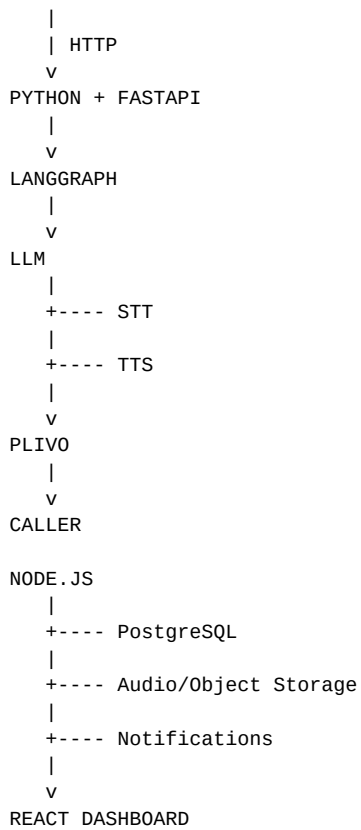
The safest order is: **LangGraph text agent** → **FastAPI AI service** → **Node/Express integration** → **React dashboard** → **telephony** → **STT/TTS** → **recording/transcript** → **summary/notifications** → **callback** → **security/testing** → **final polish**. Do not wait until the last week to integrate telephony; the Day 35 milestone should already demonstrate a real phone conversation.

33. Final Architecture Decision

```

REACT
  |
  v
NODE.JS + EXPRESS

```



This architecture is recommended as the project baseline. Optional technologies and advanced features should not be allowed to destabilize the MVP.